Лабораторная работа №3: Вероятностное динамическое программирование. Марковские процессы принятия решений.

Слёлин А.В. ПМИ-11МО

30 марта, 2026 г.

Постановка задачи

Рассматривается задача управления запасами (складом) с дискретным временем. Склад имеет вместимость C единиц товара. В начале каждого дня менеджер может принять решение о заказе a единиц товара ($0 \leqslant a \leqslant C - s$, где s — текущий запас). Спрос d в течение дня является случайной величиной с заданным распределением вероятностей. Если спрос превышает доступный товар, неудовлетворённая часть теряется (отсутствие штрафа за дефицит в базовом варианте). Цель — максимизировать ожидаемую дисконтированную прибыль за бесконечный горизонт.

Формально: найти оптимальную стратегию (политику) $\pi : S \to A$, где $S = \{0, 1, \ldots, C\}$ — множество состояний (уровень запаса), $A(s) = \{0, 1, \ldots, C - s\}$ — допустимые действия (заказ), максимизирующую

$$V(s) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \;\middle|\; s_0 = s,\ a_t = \pi(s_t)\right],$$

где $\gamma \in [0, 1)$ — коэффициент дисконтирования, $R(s, a)$ — мгновенная награда.

Мгновенная награда вычисляется по формуле:

$$R(s, a, d) = p \cdot \min(d, s + a) - c \cdot a - h \cdot \max(s + a - d, 0),$$

а затем усредняется по спросу d: $R(s, a) = \sum_d P(d) R(s, a, d)$. Здесь p — цена продажи, c — стоимость заказа, h — штраф за хранение.

В рамках варианта 3 заданы следующие параметры (MDP с неопределённостью спроса):

- Вместимость $C = 5$.
- Цена продажи $p = 10$, стоимость заказа $c = 6$, штраф за хранение $h = 1$, штраф за дефицит отсутствует.
- Дисконт $\gamma = 0.9$.
- Спрос рассчитывается по формуле: $P(d = k) = e^{-\lambda} \cdot \dfrac{\lambda^k}{k!}$, причём для 5 вероятность известна: $P(d = 0) = 0.2231,\ P(d = 1) = 0.3347,\ P(d = 2) = 0.2510,\ P(d = 3) = 0.1255,\ P(d = 4) = 0.0471,\ P(d = 5) = 0.0141$.
- Начальное состояние $s_0 = C/2 = 3$.

Требуется вычислить оптимальную политику $\pi(s)$ (сколько заказывать при каждом уровне запаса) и ценность начального состояния $V(3)$.

Теоретическая часть

Марковский процесс принятия решений (MDP)

MDP описывается кортежем $\langle S, A, P, R, \gamma \rangle$:

- S — конечное множество состояний.
- A — конечное множество действий; $A(s)$ — допустимые действия в состоянии s.
- $P(s'|s,a)$ — вероятность перехода в состояние s' после выполнения действия a в состоянии s.
- $R(s,a)$ — ожидаемая мгновенная награда.
- γ — коэффициент дисконтирования.

Оптимальная функция ценности $V^*(s)$ удовлетворяет уравнению Беллмана:

$$V^*(s) = \max_{a \in A(s)} \left[R(s,a) + \gamma \sum_{s' \in S} P(s'|s,a) V^*(s') \right].$$

Алгоритм итерации по ценности (Value Iteration)

Это итерационный метод решения уравнения Беллмана:

1. Инициализировать $V(s) = 0$ для всех s.
2. Повторять:

$$V_{\text{new}}(s) = \max_a \left[R(s,a) + \gamma \sum_{s'} P(s'|s,a) V(s') \right]$$

   для всех s, пока максимальное изменение $\Delta = \max_s |V_{\text{new}}(s) - V(s)|$ не станет меньше заданной точности ε.

3. Извлечь оптимальную политику:

$$\pi(s) = \arg\max_a \left[R(s,a) + \gamma \sum_{s'} P(s'|s,a) V(s') \right].$$

Сложность одной итерации $O(|S|^2|A|)$.

Алгоритм итерации по политике (Policy Iteration)

Этот метод часто сходится быстрее:

1. Начать с произвольной политики π_0.
2. **Оценка политики**: решить систему линейных уравнений
$$V^\pi(s) = R(s, \pi(s)) + \gamma \sum_{s'} P(s'|s, \pi(s)) V^\pi(s')$$
(например, итеративно до сходимости).
3. **Улучшение политики**:
$$\pi_{\text{new}}(s) = \arg\max_a \left[R(s,a) + \gamma \sum_{s'} P(s'|s,a) V^\pi(s') \right].$$
4. Если $\pi_{\text{new}} = \pi$, остановиться; иначе повторить с шага 2.

В данной работе оба алгоритма реализованы на языке Java.

Алгоритм (реализация)

Программа состоит из двух основных классов:

- `MDP` — хранит параметры задачи, вычисляет вероятности переходов и ожидаемые награды, реализует value iteration и policy iteration.
- `Main` — обеспечивает ввод параметров пользователем, вывод результатов в цветном формате с использованием библиотеки Jansi.

Вычисление переходов и наград

Для каждого состояния s и действия a (от 0 до $C - s$) вычисляется:

$$P(s'|s,a) = \sum_{d:\, \max(0, s+a-d) = s'} P(d),$$

$$R(s,a) = \sum_d P(d) \big(p \cdot \min(d, s+a) - c \cdot a - h \cdot \max(s+a-d, 0) \big).$$

Спрос считается по формуле для пуассоновского распределения.

Value Iteration

Итерации выполняются до тех пор, пока максимальное изменение V не станет меньше $\varepsilon = 10^{-9}$ (или до 1000 итераций). На каждом шаге для каждого состояния перебираются все допустимые действия, вычисляется значение по формуле Беллмана и выбирается максимум.

Policy Iteration

Начальная политика: всегда заказывать 0. Затем в цикле:

1. Оценка политики итеративным методом (аналогично value iteration, но без максимизации, с фиксированным действием).
2. Улучшение: для каждого состояния перебираются все действия и выбирается лучшее относительно текущей V.

Процесс повторяется, пока политика не стабилизируется.

Восстановление политики

После сходимости value iteration оптимальная политика извлекается путём выбора действия, максимизирующего правую часть уравнения Беллмана с финальной V.

Код

Листинг 1: Главный класс Main.java

```java
package a.slelin.work.algorithms.dp.laba3;

import org.fusesource.jansi.Ansi;

import java.util.List;
import java.util.Scanner;

public class Main {

    private static final Scanner scanner = new Scanner(System.in);

    public static final List<String> EXIT_KEY_WORDS = List.of(
            "\\exit", "\\quite", "\\stop", "\\end", "\\close",
            "\\выйти", "\\выход", "\\завершить", "\\стоп", "\\конец"
    );

    public static final List<String> YES_ANSWERS = List.of(
            "yes", "y", "да", "д"
    );

    public static final List<String> NO_ANSWERS = List.of(
            "no", "n", "нет", "н"
    );
```

```java

    static void main() {

        System.out.println(Ansi.ansi().fgRgb(255, 122, 247).a("Марковские процессы с пуассоновским спросом.\n").reset());
        delimiter();

        while (true) {

            int C = readInt("Введите максимальную вместимость склада C (по умолчанию 5): ", 5, 1, 5);
            double price = readDouble("Введите цену продажи за единицу (по умолчанию 12): ", 12, 0, Double.MAX_VALUE);
            double cost = readDouble("Введите стоимость заказа за единицу (по умолчанию 6): ", 6, 0, Double.MAX_VALUE);
            double holding = readDouble("Введите штраф за хранение за единицу (по умолчанию 1): ", 1, 0, Double.MAX_VALUE);
            double shortage = readDouble("Введите штраф за дефицит за единицу (по умолчанию 0): ", 0, 0, Double.MAX_VALUE);
            double gamma = readDouble("Введите коэффициент дисконтирования (0 < 1, по умолчанию 0.9): ", 0.9, 0, 1);
            double lambda = readDouble("Введите параметр распределения Пуассона (по умолчанию 1.5): ", 1.5, 0.001, Double.MAX_VALUE);
            int startState = readInt("Введите начальный уровень запаса (0.." + C + ", по умолчанию " + (C / 2) + "): ", C / 2, 0, C);

            delimiter();
            System.out.println(Ansi.ansi().fgBlue().a("Итог:").reset());
            System.out.println(Ansi.ansi().fgBlue().a("\tВместимость склада C = " + C).reset());
            System.out.println(Ansi.ansi().fgBlue().a("\tЦена продажи price = " + price).reset());
            System.out.println(Ansi.ansi().fgBlue().a("\tСтоимость заказа cost = " + cost).reset());
            System.out.println(Ansi.ansi().fgBlue().a("\tШтраф за хранение holding = " + holding).reset());
            System.out.println(Ansi.ansi().fgBlue().a("\tШтраф за дефицит shortage = " + shortage).reset());
```

```java
System.out.println(Ansi.ansi().fgBlue().a("\tКоэффициент дисконтирования    = " + gamma).reset());
System.out.println(Ansi.ansi().fgBlue().a("\tПараметр распределения Пуассона    = " + gamma).reset());
System.out.println(Ansi.ansi().fgBlue().a("\tНачальный уровень запаса startStage = " + gamma).reset());
delimiter();

System.out.println(Ansi.ansi().fgBlue().a("Считаем распределение спроса...").reset());
MDP mdp = new MDP(C, price, cost, holding, shortage, gamma, lambda);

double[] probs = mdp.getDemandProbs();
int maxDemand = mdp.getMaxDemand();

int colWidth = 10;
for (int d = 0; d <= maxDemand; d++) {
    String probStr = String.format("%.6f", probs[d]);
    colWidth = Math.max(colWidth, probStr.length());
}
colWidth = Math.max(colWidth, "Вероятность".length());

System.out.printf("%-12s", Ansi.ansi().fgGreen().a("Спрос (d)").reset());
for (int d = 0; d <= maxDemand; d++) {
    System.out.print(Ansi.ansi().fgCyan().a(("%" + colWidth + "d").formatted(d)).reset());
}
System.out.println();

System.out.printf("%-12s", Ansi.ansi().fgGreen().a("Вероятность (p)").reset());
for (int d = 0; d <= maxDemand; d++) {
    System.out.print(Ansi.ansi().fg(Ansi.Color.MAGENTA).a(("%" + colWidth + ".6f").formatted(probs[d])).reset());
}
System.out.println();
delimiter();

System.out.println(Ansi.ansi().fgBlue().a("Считаем функцию ценности V(s)...").reset());
```

```java
        double[] V = mdp.valueIteration();

        System.out.println(Ansi.ansi().fgGreen().a("Состояние (запас) | Ценность").reset());
        for (int s = 0; s <= C; s++) {
            System.out.print(Ansi.ansi().fgCyan().a(("%-18d").formatted(s)).reset());
            System.out.print(" | ");
            System.out.println(Ansi.ansi().fg(Ansi.Color.MAGENTA).a(("%.6f").formatted(V[s])).reset());
        }
        delimiter();

        System.out.println(Ansi.ansi().fgBlue().a("Считаем оптимальная политику   (s)...").reset());
        int[] policy = mdp.extractPolicy(V);

        System.out.println(Ansi.ansi().fgGreen().a("Состояние (запас) | Оптимальный заказ | Запас после заказа").reset());
        for (int s = 0; s <= C; s++) {
            int a = policy[s];
            System.out.print(Ansi.ansi().fgCyan().a(("%-18d").formatted(s)).reset());
            System.out.print(" | ");
            System.out.print(Ansi.ansi().fg(Ansi.Color.MAGENTA).a(("%-17d").formatted(a)).reset());
            System.out.print(" | ");
            System.out.println(Ansi.ansi().fgRgb(250, 205, 82).a(("%d").formatted(s + a)).reset());
        }
        delimiter();

        System.out.println(Ansi.ansi().fgGreen().a("Ценность начального состояния %d : %.6f".formatted(startState, V[startState])).reset());

        double[] Vpi = mdp.policyIteration();
        System.out.println(Ansi.ansi().fgGreen().a("Ценность начального состояния (Policy Iteration) %d : %.6f".formatted(startState, Vpi[startState])).reset());
        delimiter();

        while (true) {
            System.out.println("Хотите попробовать ещё один
```

```
                    пример?");
                String input = scanner.nextLine().trim();

                if (isWantExit(input) || NO_ANSWERS.contains(input.toLowerCase())) {
                    System.exit(0);
                } else if (YES_ANSWERS.contains(input.toLowerCase())) {
                    delimiter();
                    break;
                } else {
                    error("не смог разобрать ваш ответ");
                }
            }

        }
    }

    private static int readInt(String prompt, int defaultValue, int min, int max) {
        System.out.print(prompt);
        String input = scanner.nextLine().trim();

        if (input.isBlank()) {
            System.out.println(Ansi.ansi().fgBlue().a("Используется значение по умолчанию: " + defaultValue)
                    .reset());
            return defaultValue;
        }

        try {

            if (isWantExit(input)) {
                System.exit(0);
            }

            int value = Integer.parseInt(input);

            if (value < min) {
                error("значение должно быть больше, чем " + min);
                return readInt(prompt, defaultValue, min, max);
            }

            if (value > max) {
                error("значение должно быть меньше, чем " + max);
```

```
152                 return readInt(prompt, defaultValue, min, max);
153             }
154
155             return value;
156         } catch (NumberFormatException e) {
157             error("не смог преобразовать строку '%s' к числу".
                    formatted(input));
158             return readInt(prompt, defaultValue, min, max);
159         }
160     }
161
162     private static double readDouble(String prompt, double
            defaultValue, double min, double max) {
163         System.out.print(prompt);
164         String input = scanner.nextLine().trim();
165
166         if (input.isBlank()) {
167             System.out.println(Ansi.ansi().fgBlue().a("Используе
                    тся значение по умолчанию: " + defaultValue)
168                     .reset());
169             return defaultValue;
170         }
171
172         try {
173             if (isWantExit(input)) {
174                 System.exit(0);
175             }
176
177             double value = Double.parseDouble(input);
178
179             if (value < min) {
180                 error("значение должно быть больше, чем " + min)
                        ;
181                 return readDouble(prompt, defaultValue, min, max
                        );
182             }
183
184             if (value > max) {
185                 error("значение должно быть меньше, чем " + max)
                        ;
186                 return readDouble(prompt, defaultValue, min, max
                        );
187             }
188
189             return value;
190         } catch (NumberFormatException e) {
191             error("не смог преобразовать строку '%s' к числу".
```

```
                formatted(input));
            return readDouble(prompt, defaultValue, min, max);
        }
    }

    public static boolean isWantExit(String source) {
        return source != null && EXIT_KEY_WORDS.contains(source.toLowerCase());
    }

    public static void error(String message) {
        System.out.println(Ansi.ansi().fgRed().a("Ошибка: " + message + "!").reset());
    }

    public static void delimiter() {
        System.out.println(Ansi.ansi().fgYellow()
                .a("
                    ************************************************
                    ").reset());
    }
}
```

Листинг 2: Класс MDP.java

```
package a.slelin.work.algorithms.dp.laba3;

import lombok.Getter;

@Getter
public class MDP {

    // Точность
    public final double epsilon = 1e-9;

    private final int C;                // запас на складе (максимальное число)
    private final double price;                 // цена продажи за единицу
    private final double cost;                  // стоимость заказа за единицу
    private final double holding;               // штраф за хранение за единицу
    private final double shortage;              // штраф за дефицит за единицу
    private final double gamma;                 // коэффициент дисконтирования
```

```java
    private final double lambda;                // параметр распределения Пуассона

    private int maxDemand;                      // максимальное значение спроса, учитываемое в расчётах
    private double[] demandProbs;               // вероятности спроса (индекс i = d)
    private final int numStates;                    // количество состояний = C + 1 (ещё нулевое)
    private final double[][][] transitions;         // transitions[s][a][sp] = вероятность перехода
    private final double[][] expectedReward;        // expectedReward[s][a] = ожидаемая мгновенная награда

    public MDP(int C, double price, double cost, double holding,
               double shortage, double gamma, double lambda) {
        this.C = C;
        this.price = price;
        this.cost = cost;
        this.holding = holding;
        this.shortage = shortage;
        this.gamma = gamma;
        this.lambda = lambda;
        this.numStates = C + 1;

        // Вычисляем вероятности спроса по Пуассону
        computeDemandProbabilities();

        // Инициализируем массивы переходов и ожидаемых наград
        transitions = new double[numStates][numStates][numStates];
        expectedReward = new double[numStates][numStates];

        // Заполняем их
        computeTransitionsAndRewards();
    }

    /**
     * Вычисляет вероятности спроса по формуле Пуассона.
     */
    private void computeDemandProbabilities() {
        maxDemand = 5;
        demandProbs = new double[maxDemand + 1];
        double sum = 0.0;
        for (int d = 0; d <= 5; d++) {
            demandProbs[d] = poissonProbability(d, lambda);
            sum += demandProbs[d];
```

```java
        }

        // Нормализация
        for (int d = 0; d <= 5; d++) {
            demandProbs[d] /= sum;
        }
    }

    private double poissonProbability(int k, double lambda) {
        return Math.exp(-lambda) * Math.pow(lambda, k) / factorial(k);
    }

    private long factorial(int n) {
        long fact = 1;
        for (int i = 2; i <= n; i++) {
            fact *= i;
        }
        return fact;
    }

    /**
     * Для каждого состояния s и действия a (0..C-s) вычисляет:
     * - вероятности перехода в sp (новое состояние)
     * - ожидаемую мгновенную награду
     */
    private void computeTransitionsAndRewards() {
        // Для каждого состояния (сколько есть сейчас на складе)
        for (int s = 0; s < numStates; s++) {
            // Для каждого действия (сколько заказываем дополнительное на склад)
            for (int a = 0; a <= C - s; a++) {

                // Для каждого возможного спроса d (сколько у нас купят по распределению)
                // Разные спросы могут приводить возможно к одному и тому же новому состоянию sp (поэтому +=)
                for (int d = 0; d <= maxDemand; d++) {

                    // Вероятность такого спроса
                    double probD = demandProbs[d];

                    // Новое состояние
                    int sp = s + a - d;

                    // Неудовлетворенный спрос
```

```java
                    if (sp < 0) sp = 0;

                    // Вероятность перехода
                    transitions[s][a][sp] += probD;

                    // Мгновенная награда при данном спросе
                    double reward = computeReward(s, a, d);
                    expectedReward[s][a] += probD * reward;
                }
            }
        }
    }

    /**
     * Вычисляет награду за один шаг: s -> a при реализации спроса d.
     */
    private double computeReward(int s, int a, int d) {
        int available = s + a; // доступно
        int sold = Math.min(d, available); // продали
        int leftover = Math.max(available - d, 0); // осталось
        int unsatisfied = Math.max(d - available, 0); // не удовлетворили

        return price * sold - cost * a - holding * leftover - shortage * unsatisfied;
    }

    /**
     * Итерация по ценности.
     */
    public double[] valueIteration() {
        double[] V = new double[numStates];
        double[] Vnew = new double[numStates];
        int maxIter = 1000;
        int iter = 0;
        double delta; // разница между векторами (чтобы когда будет маленькая выйти)

        do {
            delta = 0.0;
            for (int s = 0; s < numStates; s++) {
                double best = -Double.MAX_VALUE;
                for (int a = 0; a <= C - s; a++) {
                    double value = expectedReward[s][a];

                    for (int sp = 0; sp < numStates; sp++) {
```

```
                        value += gamma * transitions[s][a][sp] * V[sp];
                    }

                    if (value > best) {
                        best = value;
                    }
                }
                Vnew[s] = best;
                delta = Math.max(delta, Math.abs(Vnew[s] - V[s]));
            }

            // переставляем вектора
            double[] tmp = V;
            V = Vnew;
            Vnew = tmp;

            iter++;
        } while (delta >= epsilon && iter < maxIter);

        return V;
    }

    /**
     * Извлекает оптимальную политику из заданной функции ценности V.
     */
    public int[] extractPolicy(double[] V) {
        int[] policy = new int[numStates];

        for (int s = 0; s < numStates; s++) {
            double bestValue = -Double.MAX_VALUE;
            int bestAction = 0;
            for (int a = 0; a <= C - s; a++) {
                double value = expectedReward[s][a];

                for (int sp = 0; sp < numStates; sp++) {
                    value += gamma * transitions[s][a][sp] * V[sp];
                }

                if (value > bestValue) {
                    bestValue = value;
                    bestAction = a;
                }
            }
```

```java
            policy[s] = bestAction;
        }
        return policy;
    }

    public int[] extractPolicy() {
        return extractPolicy(valueIteration());
    }

    /**
     * Итерация по политике (Policy Iteration).
     */
    public double[] policyIteration() {
        // Начальная политика: всегда заказывать 0
        int[] policy = new int[numStates];

        boolean changed;
        do {
            // Оценка политики: решаем систему линейных уравнений V = R +    * P * V
            double[] V = evaluatePolicy(policy);

            // Улучшение политики
            changed = false;
            for (int s = 0; s < numStates; s++) {
                double bestValue = -Double.MAX_VALUE;
                int bestAction = policy[s];
                for (int a = 0; a <= C - s; a++) {
                    double value = expectedReward[s][a];
                    for (int sp = 0; sp < numStates; sp++) {
                        value += gamma * transitions[s][a][sp] * V[sp];
                    }
                    if (value > bestValue) {
                        bestValue = value;
                        bestAction = a;
                    }
                }
                if (bestAction != policy[s]) {
                    policy[s] = bestAction;
                    changed = true;
                }
            }
        } while (changed);

        // После сходимости возвращаем ценности для финальной политики
```

```java
        return evaluatePolicy(policy);
    }

    /**
     * Оценка заданной политики: решаем систему (I -   *P) V = R.
     * Используется итерационный метод (оценка политики итеративно).
     * Для простоты реализуем итерации до сходимости.
     */
    private double[] evaluatePolicy(int[] policy) {
        double[] V = new double[numStates];
        double[] Vnew = new double[numStates];
        double epsilon = 1e-6;
        int maxIter = 1000;
        int iter = 0;
        double delta;

        do {
            delta = 0.0;
            for (int s = 0; s < numStates; s++) {
                int a = policy[s];
                double value = expectedReward[s][a];
                for (int sp = 0; sp < numStates; sp++) {
                    value += gamma * transitions[s][a][sp] * V[sp];
                }
                Vnew[s] = value;
                delta = Math.max(delta, Math.abs(Vnew[s] - V[s]));
            }
            double[] tmp = V;
            V = Vnew;
            Vnew = tmp;
            iter++;
        } while (delta >= epsilon && iter < maxIter);
        return V;
    }
}
```

Пример работы

Запуск программы с параметрами (пуассоновский спрос) даёт следующие результаты.

Параметры:

- Вместимость склада $C = 5$.
- Цена продажи $p = 12$, стоимость заказа $c = 6$, штраф за хранение $h = 1$, штраф за дефицит отсутствует.
- Коэффициент дисконтирования $\gamma = 0.9$.
- Спрос распределён по закону Пуассона с параметром $\lambda = 1.5$. Вероятности:

d	$P(d)$
0	0.2231
1	0.3347
2	0.2510
3	0.1255
4	0.0471
5	0.0141

Функция ценности $V(s)$:

s	$V(s)$
0	60.38
1	66.38
2	72.38
3	78.16
4	82.66
5	86.16

Ценность начального состояния $s_0 = 3$: $V(3) \approx 78.16$.

Оптимальная политика $\pi(s)$ (сколько заказывать):

s	a	$s + a$
0	2	2
1	1	2
2	0	2
3	0	3
4	0	4
5	0	5

Интерпретация: При пуассоновском спросе с $\lambda = 1.5$ средний спрос выше, чем в варианте 1, поэтому оптимально поддерживать больший запас. При пустом складе заказывается 2 единицы, при остатке 1 — 1. При остатке 2 и выше заказ не производится. Ценность начального состояния $V(3) = 78.16$ — ожидаемая дисконтированная прибыль при старте с трёх единиц товара.

Тестирование

Листинг 3: Базовые тесты

```java
package a.slelin.work.algorithms.dp.laba3.test;

import a.slelin.work.algorithms.dp.laba3.MDP;
import org.junit.jupiter.api.Test;

import java.util.Arrays;

public class SimpleTest {

    @Test
    public void test1() {
        MDP mdp = new MDP(5, 12, 6, 1, 0, 0.9, 1.5);
        double[] valueIteration = mdp.valueIteration();
        System.out.println(valueIteration[3]);
        int[] policy = mdp.extractPolicy();
        System.out.println(Arrays.toString(policy));
    }

    @Test
    public void test2() {
        MDP mdp = new MDP(5, 12, 6, 1, 0, 0.9, 0.5);
        double[] valueIteration = mdp.valueIteration();
        System.out.println(valueIteration[3]);
        int[] policy = mdp.extractPolicy();
        System.out.println(Arrays.toString(policy));
    }

    @Test
    public void test3() {
        MDP mdp = new MDP(5, 12, 6, 1, 0, 0.9, 3.0);
        double[] valueIteration = mdp.valueIteration();
        System.out.println(valueIteration[3]);
        int[] policy = mdp.extractPolicy();
        System.out.println(Arrays.toString(policy));
    }
}
```

Все тесты пройдены!

Выводы

В ходе выполнения лабораторной работы были изучены методы вероятностного динамического программирования для задач управления в условиях неопределённости. Реализованы алгоритмы итерации по ценности и итерации по политике для марковского процесса принятия решений на примере задачи управле-

ния запасами. Освоены понятия состояний, действий, вероятностей переходов, функции награды и дисконтирования. Проведён анализ влияния параметров (дисконт, штраф за хранение) на оптимальную политику. Полученные навыки могут быть применены для решения широкого класса задач оптимального управления в стохастических средах, включая управление запасами, ценообразование, управление очередями и т.д.